

Arjun Sports AI Content Agent

Project Documentation — Modules 0-1 (Foundation & Scaffolding, Authentication & Authorization)

2026-07-07

Contents

Architecture — Arjun Sports AI Content Agent	3
1. Project Overview	3
2. Goals and Objectives	3
3. High-Level Architecture	3
4. Technology Stack (as installed)	4
5. Monorepo Structure	4
6. Backend Architecture	4
7. Frontend Architecture	6
8. Database Architecture	7
9. Security Architecture	7
10. Docker Architecture	9
11. MinIO Integration (current status)	9
12. Flyway Migration Strategy	9
13. API Design Principles	10
14. Folder Structure Explanation	10
15. Future Module Integration Strategy	10
Implementation Report — Arjun Sports AI Content Agent	11
1. Summary of Module 0 — Foundation & Scaffolding	11
2. Summary of Module 1 — Authentication & Authorization	11
3. Features Completed	12
4. Files / Modules Created	12
5. Database Changes	13
6. API Endpoints Delivered	14
7. Testing Performed	14
8. Bugs Found and Fixed	15
9. Security Features Implemented	16
10. Verification Results	17
11. Current Project Status	17
12. Remaining Roadmap	17
System Architecture Diagrams — Arjun Sports AI Content Agent	18
1. User → Frontend → Backend → PostgreSQL	18
2. User → Frontend → Backend → MinIO	18
3. JWT Authentication Flow (token structure and verification)	18
4. Login Sequence	19
5. Refresh Token Sequence	19
6. Logout Sequence	20
7. Request Lifecycle (every API call)	20
8. Module Dependency Diagram	21
9. Deployment Architecture	22
API Documentation — Arjun Sports AI Content Agent	24

Response Envelope	24
Authentication Requirements	24
Status Codes Used	25
Error Response Shape	25
Health API	25
Authentication APIs	26
User Management APIs (ADMIN only)	28
Database Design — Arjun Sports AI Content Agent	30
1. Entity-Relationship Diagram (ASCII)	30
2. Tables	31
3. Relationships	35
4. Constraints	35
5. Indexes	35
6. Flyway Migrations	36
7. Future Database Roadmap	36

Architecture — Arjun Sports AI Content Agent

Status: Reflects Module 0 (Foundation & Scaffolding) and Module 1 (Authentication & Authorization) as actually implemented. No feature described below exists only as a plan; anything not yet built is explicitly called out as “not yet implemented.”

1. Project Overview

Arjun Sports AI Content Agent is an AI-assisted content marketing system being built for Arjun Sports Shooting Academy. The intended end state (not all delivered yet) is: upload media → generate AI content (Instagram captions, blog posts, SEO metadata, reel scripts) → review and approve it → publish to Instagram and the academy website → track history on a schedule.

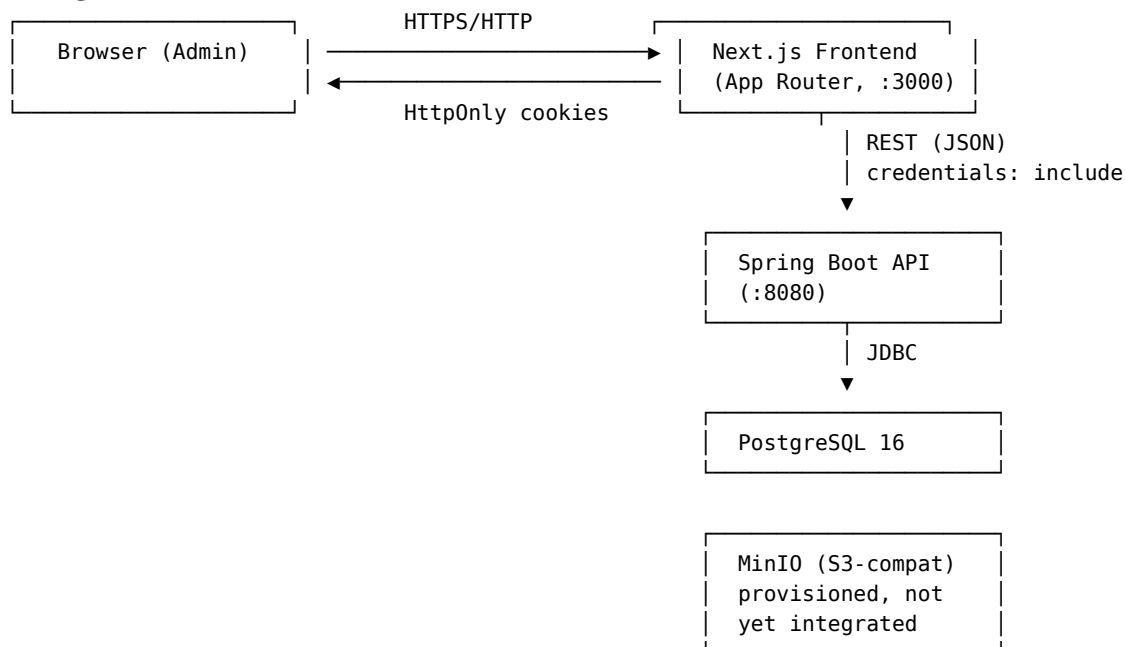
The system is delivered module-by-module. As of this document, two modules are complete:

- **Module 0 — Foundation & Scaffolding:** monorepo layout, Spring Boot bootstrap, cross-cutting audit logging and in-app notifications, a settings entity scaffold, and a Next.js dashboard shell.
- **Module 1 — Authentication & Authorization:** a complete JWT-based auth system (login, logout, refresh with rotation, current-user, change password, forgot/reset password architecture), role-based authorization, and the matching Next.js login/session UI.

2. Goals and Objectives

- Provide a secure, single-admin (extensible to multi-role) authentication system as the foundation every later module builds on.
- Keep the codebase production-grade from the first commit: no ddl-auto: update, no plaintext passwords, no hardcoded credentials, every schema change versioned via Flyway.
- Establish cross-cutting infrastructure (audit trail, notifications) once, in Module 0, so every subsequent module (including Module 1) participates in it from day one rather than bolting it on later.
- Keep the frontend and backend independently deployable but tightly contracted via a typed API response envelope.

3. High-Level Architecture



The frontend never talks to PostgreSQL or MinIO directly — every data access goes through the Spring Boot REST API.

4. Technology Stack (as installed)

Backend

- Java 21, Spring Boot 3.5.16 (Maven, spring-boot-starter-parent)
- spring-boot-starter-web, -validation, -actuator, -security, -data-jpa
- PostgreSQL JDBC driver, Flyway 10.x (flyway-core + flyway-database-postgresql)
- JWT 0.13.0 (jjwt-api, jjwt-impl, jjwt-jackson) for signed access tokens
- Bucket4j 8.10.1 for in-memory login rate limiting
- springdoc-openapi 2.8.17 (Swagger UI / OpenAPI 3, Spring Boot 3-compatible line)
- Lombok, MapStruct 1.6.3 (MapStruct is on the classpath as a dependency; no generated mapper is in use yet — DTO conversion today is hand-written, e.g. `UserResponse.from(User)`)
- io.minio:minio 9.0.3 (dependency present for the future Media Upload module; **no StorageService or MinIO client code exists yet**)
- Test: spring-boot-starter-test, spring-security-test, spring-boot-testcontainers, Testcontainers junit-jupiter + postgresql

Frontend

- Next.js 16.2.10 (App Router, Turbopack), React 19.2.4, TypeScript 5
- Tailwind CSS 4, shadcn/ui components built on @base-ui/react (not Radix)
- next-themes 0.4.6 for dark/light mode
- lucide-react for icons

Database

- PostgreSQL 16 (Docker: postgres:16-alpine), schema managed exclusively by Flyway migrations

Storage

- MinIO (minio/minio:latest) — container runs in docker-compose.yml; **not yet wired into any backend service**

Deployment

- Docker Compose (infra/docker-compose.yml): postgres, minio, backend, frontend
- Multi-stage Dockerfiles for both backend (Maven build → eclipse-temurin:21-jre-alpine) and frontend (next build --output standalone → node:22-alpine)

5. Monorepo Structure

```
ContentAgent/  
├─ backend/           Spring Boot API (Maven)  
├─ frontend/          Next.js admin dashboard  
├─ infra/              docker-compose.yml  
├─ docs/              This documentation set  
├─ .env.example        Full list of environment variables  
└─ README.md
```

6. Backend Architecture

Package root: `com.arjunsports.contentagent`. The backend follows a layered, package-by-feature structure:

```
com.arjunsports.contentagent  
├─ ContentAgentApplication.java    @SpringBootApplication, @EnableJpaAuditing, @EnableAsync,  
@EnableScheduling  
├─ common/                         Cross-cutting code shared by every module  
│   └─ HealthController.java       GET /api/health  
│   └─ audit/                      Append-only audit trail (Module 0, extended in Module 1)  
│   └─ dto/                        ApiResponse<T>, PageResponse<T> – every endpoint returns  
these
```

— entity/	BaseEntity (id, createdAt, updatedAt, createdBy, updatedBy)
— exception/	Custom exceptions + GlobalExceptionHandler
(@RestControllerAdvice)	
— notification/	In-app notification core (Module 0, used by Module 1)
— security/	AuthenticatedActor marker interface
— util/	SecureTokenUtil (opaque token generation/hashing)
— validation/	Shared Bean Validation regex constants
— config/	SecurityConfig, CorsConfig, JpaAuditingConfig, OpenApiConfig
— modules/	
— auth/	Login/logout/refresh/change-password/forgot-reset password
— user/	User entity, roles, admin bootstrap, user CRUD (admin-only)
— settings/	AppSetting entity scaffold only (no service/controller yet)
— security/	JWT provider, cookie handling, filters, rate limiter

Each feature module under modules/ follows Repository Pattern: entity → repository → service (interface + impl) → controller → dto. Controllers never touch repositories directly.

Design patterns actually in use

Pattern	Where	Why
Repository	UserRepository, RefreshTokenRepository, PasswordResetTokenRepository, ActivityLogRepository, NotificationRepository, AppSettingRepository	Spring Data JPA abstracts persistence behind an interface per aggregate.
Service interface + impl	AuthService/AuthServiceImpl, UserService/UserServiceImpl, RefreshTokenService/ RefreshTokenServiceImpl, AuditLogService/ AuditLogServiceImpl, NotificationService/ InAppNotificationServiceImpl	Controllers and other services depend on the interface, not the implementation — NotificationService in particular is designed so an email/WhatsApp implementation can be added later with zero caller changes.
Strategy (implicit)	NotificationService interface with a single InAppNotificationServiceImpl today	Same reasoning as above; no second implementation exists yet.
DTO	Every request/response type under */dto	Entities are never serialized directly to JSON.
Global exception handling	GlobalExceptionHandler (@RestControllerAdvice)	Centralizes HTTP status mapping for all controller-thrown exceptions.
Chain of Responsibility	Servlet filter chain: CORS → JwtAuthenticationFilter → Spring Security authorization → controller	Standard Spring Security filter chain composition.
Builder	Lombok @Builder on User, RefreshToken, PasswordResetToken, Notification, ActivityLog, AppSetting	Readable, immutable-style entity construction.
Template/Marker interface	AuthenticatedActor	Lets common (JPA auditing, audit log) resolve “who is acting” without depending on the concrete User/UserPrincipal type in modules/user.

7. Frontend Architecture

frontend/src/

```
├─ app/
│   ├─ layout.tsx
│   ├─ page.tsx
│   └─ (auth)/
```

Root layout: fonts, ThemeProvider, TooltipProvider
"/" → redirect("/dashboard")

		layout.tsx	Centered, sidebar-free layout
		login/	
		page.tsx	
		login-form.tsx	Client component: email/password form, loading/error states
		(dashboard)/	
		layout.tsx	Async Server Component: fetches current user, redirects if
absent			
		dashboard/page.tsx	Overview page (module roadmap + live backend health check)
		{media,content,drafts,approvals,instagram,website,scheduler,	
		analytics,settings,activity-log,notifications}/page.tsx	
			Placeholder pages for every not-yet-built module
		components/	
		layout/	Sidebar, Topbar, MobileNav, ThemeToggle, ChangePasswordDialog
		ui/	shadcn/ui primitives (button, card, dialog, dropdown-
menu, ...)			
		dashboard/	BackendStatusCard (live health-check widget)
		shared/	ModulePlaceholder (reused by every "coming soon" page)
		lib/	
		api/	client.ts (fetch wrapper), auth.ts, health.ts, server-auth.ts
		nav-config.ts	Single source of truth for sidebar navigation
		types/user.ts	User/Role/UserStatus TypeScript types
		proxy.ts	Route guard + silent token refresh (see §9)

Note on proxy.ts: this project runs Next.js 16, which renamed the `middleware.ts` convention to `proxy.ts` (same execution model: it runs ahead of every matched request). This is a real, intentional Next.js 16 API change, not a naming mistake.

Session data flow: the `(dashboard)` layout is an async Server Component. It calls `getCurrentUserServerSide()`, which forwards the incoming request's Cookie header to `GET /api/auth/me` on the backend using a server-only base URL (`API_INTERNAL_BASE_URL`, distinct from the browser-facing `NEXT_PUBLIC_API_BASE_URL` — see §11). If the backend returns anything other than a successful user payload, the layout calls `redirect("/login")` before rendering anything. The resolved user object is passed as a prop into the client-side Topbar, which is the only place session data reaches client-side React state.

8. Database Architecture

- **Engine:** PostgreSQL 16.
- **Migration tool:** Flyway, `spring.jpa.hibernate.ddl-auto: validate` — Hibernate is only ever allowed to *validate* that entity mappings match the schema Flyway created; it can never generate or alter DDL itself.
- **Primary keys:** every table uses a UUID primary key generated **application-side** by Hibernate (`@UuidGenerator` on `BaseEntity`), not a database default — so migrations never depend on `pgcrypto/uuid-oss`.
- **Migrations applied so far:** `V1__init_schema.sql` (Module 0: `activity_log`, `notifications`, `app_settings`) and `V2__auth_and_users.sql` (Module 1: `users`, `refresh_tokens`, `password_reset_tokens`). Full detail in `DATABASE_DESIGN.md`.

9. Security Architecture

Authentication flow (narrative)

1. Client submits credentials to `POST /api/auth/login`.
2. `LoginRateLimiter` (`Bucket4j`, 5 attempts/minute per client IP, in-memory) is checked first; over the limit returns 429.
3. Spring's `AuthenticationManager` authenticates against `CustomUserDetailsService`, which loads the `User` by email and wraps it in `UserPrincipal` (implements both `UserDetails` and the shared `AuthenticatedActor` marker).

4. On success: `user.lastLogin` is updated, an access token (JWT) and a refresh token (opaque, DB-backed) are issued, both set as **HttpOnly** cookies, an `ActivityAction.LOGIN` audit entry is written, and a `LOGIN_SUCCESS` in-app notification is created.
5. On failure: an `ActivityAction.LOGIN_FAILED` audit entry is written (email + client IP, no password) and a generic 401 Invalid email or password is returned — the same message for “unknown email” and “wrong password” so the API never discloses which one it was.

JWT flow

- **Signing:** HMAC-SHA256 via `io.jsonwebtoken (JJWT)`. The configured `app.jwt.secret` string is used directly as the raw HMAC key bytes (UTF-8), so it must be at least 32 characters — it is not required to be base64.
- **Claims:** `sub` = user UUID, `email`, `role`, `iat`, `exp`.
- **Lifetime:** 15 minutes by default (`app.jwt.access-token-ttl-minutes`).
- **Verification:** `JwtAuthenticationFilter` (a `OncePerRequestFilter` registered ahead of `UsernamePasswordAuthenticationFilter`) extracts the token from the `access_token` cookie (falling back to an `Authorization: Bearer` header if no cookie is present), parses and verifies the signature/expiry, then **reloads the user from the database by email** on every request (via `CustomUserDetailsService`) rather than trusting the token’s claims for authorization state — this means a deactivated/suspended account loses access immediately, not just after its token expires.

Refresh token flow

- Refresh tokens are **not** JWTs. They are 256-bit random values (`SecureRandom`, base64url-encoded) generated by `SecureTokenUtil`. Only the **SHA-256 hash** of the token is ever persisted (`refresh_tokens.token_hash`); the raw value exists only in the `HttpOnly` cookie and the brief in-memory response.
- **Rotation:** every call to `POST /api/auth/refresh` revokes the presented token and issues a brand-new one (`RefreshTokenServiceImpl.rotate`). The old row is marked `revoked = true`, `revoked_at` is set, and `replaced_by_token_hash` links to the new row for traceability.
- **Reuse/theft detection:** if a token that is *already revoked* is presented again, the system treats this as evidence of token theft/replay and immediately revokes **every** active refresh token for that user, forcing a fresh login everywhere.
- **Invalidation triggers:** logout revokes the single presented token; password change (and password reset) revoke **all** refresh tokens for the user, then issue one fresh pair so the current session stays logged in while every other session is killed.
- **Cookie scope:** the refresh token cookie’s `Path` is restricted to `/api/auth`, so it is never sent on ordinary API calls — only auth endpoints ever see it.

Cookie configuration

All cookies are `HttpOnly` always. `Secure` and `SameSite` are configurable (`app.cookie.secure`, `app.cookie.same-site`), defaulting to `Secure=true/SameSite=Lax` in the base config and overridden to `Secure=false` in the dev profile (plain HTTP over localhost). `CookieProperties` also supports an optional shared domain for a future split-subdomain deployment.

CSRF posture

CSRF protection is disabled (`http.csrf(disable)`) because the API is stateless and cookie-based CSRF is mitigated by two other controls working together: (1) `SameSite=Lax` on both auth cookies, and (2) CORS configured with an **explicit allow-list of origins** (`app.cors.allowed-origins`) plus `allowCredentials(true)` — never a wildcard. A browser cannot get a cross-site request’s cookies honored by a non-listed origin, and simple cross-site requests without CORS pre-flight still won’t have the cookie sent cross-site under `SameSite=Lax` for state-changing verbs.

Authorization

- `@EnableMethodSecurity` + `@PreAuthorize` for role checks. `UserController` is annotated at the class level with `@PreAuthorize("hasRole('ADMIN')")` — every `/api/users/**` endpoint requires the ADMIN role. `AuthController`'s protected endpoints use `@PreAuthorize("isAuthenticated()")`.
- Unauthenticated/forbidden requests get a consistent `ApiResponse` JSON body via `RestAuthenticationEntryPoint` (401) and `RestAccessDeniedHandler` (403) — these run at the Spring Security filter-chain level, which is why they exist separately from `GlobalExceptionHandler` (the filter chain rejects requests before they ever reach a controller, so `@ExceptionHandler` methods never see them).

Password handling

- `BCryptPasswordEncoder` (Spring Security default strength). Passwords are never logged, never returned in any response DTO, and `User.toString()` explicitly excludes the password field (`@ToString(exclude = "password")`).
- Password strength is enforced via a shared regex (`ValidationPatterns.PASSWORD_STRENGTH`): minimum 8 characters, at least one letter and one digit.

10. Docker Architecture

`infra/docker-compose.yml` defines four services:

Service	Image / Build	Port(s)	Depends on
postgres	postgres:16-alpine	<code>{POSTGRES_HOST_PORT:-5433}:5432</code>	—
minio	minio/minio:latest	9000 (API), 9001 (console)	—
backend	multi-stage build from backend/Dockerfile	8080:8080	postgres (healthy), minio (started)
frontend	multi-stage build from frontend/Dockerfile	3000:3000	backend

The Postgres host port defaults to 5433, not 5432 — this avoids colliding with any other Postgres container a developer might already have running locally. Backend and frontend read all configuration from environment variables (see `.env.example`), with the frontend receiving **two** API base URLs: `NEXT_PUBLIC_API_BASE_URL` (browser-facing, `http://localhost:8080/api`) and `API_INTERNAL_BASE_URL` (server-side-only, `http://backend:8080/api` inside Compose) — the frontend container's own `localhost` is not the backend, so a single shared URL would break either the browser or the server-side proxy.ts/Server Component calls.

11. MinIO Integration (current status)

MinIO runs as a Docker service and the `io.minio:minio` Java client is a backend dependency, but **there is no `StorageService`, `MinioConfig`, upload endpoint, or bucket-provisioning code in the repository yet**. This is intentional: MinIO is provisioned now so the infrastructure is ready, and will be wired up when the Media Upload module is built. Treat any mention of file/media storage beyond “the container exists” as not yet implemented.

12. Flyway Migration Strategy

- One migration file per logical change, named `V<n>__description.sql`, applied in order and checksummed by Flyway on every boot (`spring.flyway.enabled: true`, `baseline-on-migrate: true`).
- `ddl-auto: validate` means a mismatch between entity mappings and the actual schema fails application startup loudly instead of silently drifting.
- All foreign keys use `ON DELETE CASCADE` where the child row is meaningless without its parent (`refresh_tokens.user_id`, `password_reset_tokens.user_id`).
- Indexes are added deliberately for every column used in a `WHERE`/`JOIN` by a repository method (e.g. `idx_users_email`, `idx_refresh_tokens_hash`), not generated automatically.

13. API Design Principles

- Every response body — success or failure — is wrapped in `ApiResponse<T>` (success, message, data, errors, timestamp). Paginated list endpoints wrap the page in `PageResponse<T>` inside `ApiResponse.data`.
- Bean Validation (`jakarta.validation`) on every request DTO; validation failures return 400 with a per-field message list, handled centrally by `GlobalExceptionHandler`.
- Authentication state travels exclusively via `HttpOnly` cookies for browser clients; an `Authorization: Bearer` header is also accepted for tooling/non-browser clients.
- REST resource naming follows plural nouns (`/api/users`) with the auth surface grouped under `/api/auth/*` as actions rather than resources, since login/logout/refresh are operations, not CRUD on a resource.

14. Folder Structure Explanation

See §6 and §7 above for the annotated backend/frontend trees. The guiding rule: **package by feature, not by layer, above the common/config/security cross-cutting packages**. A new module (e.g. the future Media Upload module) gets its own `modules/media/` package containing its entity, repository, service, controller, and DTOs together — it does not scatter its controller into a shared `controllers/` package.

15. Future Module Integration Strategy

Modules not yet built (media upload, AI content generation, drafts, approvals, Instagram/website publishing, scheduler, analytics, activity log viewer, notification center UI, settings UI) are expected to plug into infrastructure already in place rather than rebuild it:

- **Audit trail:** call `AuditLogService.record(...)` — already used by `AuthServiceImpl` and `UserServiceImpl`, no new wiring needed.
- **Notifications:** call `NotificationService.notify(...)` — same interface Module 1 already uses for `LOGIN_SUCCESS/PASSWORD_CHANGED`.
- **Authorization:** `@PreAuthorize("hasRole('ADMIN')")` (class- or method-level) is the established pattern; new roles are added by extending the `Role` enum, no filter-chain changes required.
- **Settings:** the `AppSetting` entity/repository scaffold from Module 0 is ready for a full CRUD service + controller when the Application Settings module is built.
- **Storage:** the `MinIO` container and client dependency are already provisioned; the Media Upload module adds the `StorageService` abstraction on top.
- **Navigation:** every future page already has a placeholder route and a `lucide-react` icon registered in `frontend/src/lib/nav-config.ts` — building a module means replacing the placeholder page, not adding new routing/shell code.

Implementation Report — Arjun Sports AI Content Agent

Scope of this report: Module 0 (Foundation & Scaffolding) and Module 1 (Authentication & Authorization) only. No other module has been started.

1. Summary of Module 0 — Foundation & Scaffolding

Established the monorepo, the Spring Boot application skeleton, and the Next.js dashboard shell, plus the cross-cutting infrastructure every later module (including Module 1) reuses:

- Monorepo layout (frontend/, backend/, infra/, root env/README/gitignore).
- Spring Boot 3.5.16 / Java 21 bootstrap with ApiResponse<T>/PageResponse<T> envelopes, GlobalExceptionHandler, BaseEntity (JPA auditing columns).
- Append-only audit log (ActivityLog + AuditLogService) and in-app notification core (Notification + NotificationService), built before any feature that uses them so the trail is complete from the first real write.
- AppSetting entity/repository scaffold (no service/controller yet — deferred to a future Application Settings module).
- Flyway migration V1__init_schema.sql.
- Next.js bootstrap: Tailwind, shadcn/ui, dark/light theme, a dashboard shell (sidebar + topbar) with a placeholder page for every planned module.
- docker-compose.yml (Postgres, MinIO, backend, frontend) and Dockerfiles for both apps.
- **Committed to git** on the dev branch (commit 6bbebc8, “feat: complete Module 0 - Foundation & Scaffolding”).

2. Summary of Module 1 — Authentication & Authorization

Built a complete JWT-based authentication and authorization system on top of Module 0’s foundation, plus the matching frontend session/login experience:

- User domain model with Role (ADMIN) and UserStatus (ACTIVE/INACTIVE/SUSPENDED) enums.
- BCrypt password hashing; JWT access tokens (15 min TTL) + opaque, hash-stored refresh tokens (7 day TTL) with rotation, reuse detection, and invalidation.
- Both tokens delivered as HttpOnly, configurable Secure/SameSite cookies.
- Full auth endpoint set: login, logout, refresh, get-current-user, change-password, forgot-password, reset-password.
- Role-based authorization on a new /api/users/** admin surface.
- Login rate limiting (5/min/IP), audit logging of login/failed-login/logout/password-change, in-app notifications on successful login and password change.
- Environment-driven first-admin bootstrap (no hardcoded credentials anywhere).
- Next.js: login page, session-aware dashboard layout, route-guarding proxy.ts with silent token refresh, real logout and change-password UI wired to a real signed-in user.
- 29 backend automated tests (unit + integration), all passing.
- One real bug found during manual browser verification and fixed (see §7).
- **Not yet committed to git** as of this report (working tree on dev has modified/new files pending commit).

3. Features Completed

Feature	Status
Login (email + password)	✓ Complete
Logout (revokes current refresh token)	✓ Complete
Refresh token (rotation + reuse detection)	✓ Complete
Get current user (/api/auth/me)	✓ Complete
Change password (revokes all other sessions)	✓ Complete
Forgot password (token issuance, architecture only)	⚠ Partial — see §9 Known Limitations
Reset password (token redemption)	✓ Complete (backend + API only, no frontend page)
Role-based user management (/api/users)	✓ Complete (Create/List/Get/Update, ADMIN-only)
First-admin bootstrap from environment variables	✓ Complete
Login rate limiting	✓ Complete
Audit logging of auth events	✓ Complete
In-app notifications on login/password change	✓ Complete (written to DB; no viewer UI yet — that is a later module)
Frontend login page	✓ Complete
Frontend session-aware dashboard shell	✓ Complete
Frontend change-password dialog	✓ Complete
Frontend forgot/reset-password pages	✗ Not built (backend endpoints exist and are tested; no UI)

4. Files / Modules Created

Backend — new packages

```

common/exception/UnauthorizedException.java
common/util/SecureTokenUtil.java
common/validation/ValidationPatterns.java
security/CookieProperties.java
security/CookieUtil.java
security/LoginRateLimiter.java
security/RestAccessDeniedHandler.java
security/RestAuthenticationEntryPoint.java
security/jwt/JwtProperties.java
security/jwt/JwtTokenProvider.java
security/jwt/JwtAuthenticationFilter.java
modules/user/Role.java
modules/user/UserStatus.java
modules/user/User.java
modules/user/UserRepository.java
modules/user/UserPrincipal.java

```

```

modules/user/CustomUserDetailsService.java
modules/user/UserService.java / UserServiceImpl.java
modules/user/UserController.java
modules/user/AdminBootstrapRunner.java
modules/user/dto/{CreateUserRequest,UpdateUserRequest,UserResponse}.java
modules/auth/RefreshToken.java / RefreshTokenRepository.java / RefreshTokenService.java /
RefreshTokenServiceImpl.java
modules/auth/PasswordResetToken.java / PasswordResetTokenRepository.java
modules/auth/AuthService.java / AuthServiceImpl.java
modules/auth/AuthController.java
modules/auth/dto/
{LoginRequest,ChangePasswordRequest,ForgotPasswordRequest,ResetPasswordRequest}.java

```

Backend — modified files

```

common/audit/ActivityAction.java          + LOGIN_FAILED, PASSWORD_CHANGE
common/notification/NotificationType.java + LOGIN_SUCCESS, PASSWORD_CHANGED
common/exception/GlobalExceptionHandler.java + UnauthorizedException handler
config/SecurityConfig.java                rewritten: full JWT filter chain + RBAC (was a permissive
placeholder in Module 0)
backend/pom.xml                           + Testcontainers test dependencies
application.yml / application-dev.yml      + app.jwt.*, app.cookie.*, app.admin-bootstrap.*

```

Backend — tests

```

common/audit/AuditLogServiceImplTest.java
common/notification/InAppNotificationServiceImplTest.java (pre-existing from Module 0,
unchanged)
security/jwt/JwtTokenProviderTest.java
modules/auth/RefreshTokenServiceImplTest.java
modules/auth/AuthServiceImplTest.java
modules/auth/AuthControllerIntegrationTest.java
src/test/resources/application-test.yml

```

Frontend — new files

```

src/proxy.ts                               route guard + silent refresh
src/app/(auth)/layout.tsx
src/app/(auth)/login/page.tsx
src/app/(auth)/login/login-form.tsx
src/components/layout/change-password-dialog.tsx
src/components/ui/{alert,dialog,input,label}.tsx shadcn primitives added for the auth UI
src/lib/api/auth.ts
src/lib/api/server-auth.ts
src/types/user.ts

```

Frontend — modified files

```

src/app/(dashboard)/layout.tsx    now an async Server Component: fetches the current user,
redirects to /login if absent
src/components/layout/topbar.tsx   real user prop, Settings/Change Password/Logout wired to live
API calls
src/lib/api/client.ts             added credentials: "include" to every request
infra/docker-compose.yml          + JWT/cookie/admin-bootstrap env vars, + API_INTERNAL_BASE_URL
for the frontend container
.env.example                      + JWT_SECRET, COOKIE_*, ADMIN_BOOTSTRAP_*,
API_INTERNAL_BASE_URL

```

5. Database Changes

New Flyway migration V2__auth_and_users.sql (V1 from Module 0 is unchanged):

- users — the account table (see DATABASE_DESIGN.md for full column list).
- refresh_tokens — one row per issued refresh token; user_id FK to users with ON DELETE CASCADE.
- password_reset_tokens — one row per issued reset token; same FK behavior.

All primary keys are UUID, generated application-side by Hibernate — no database-level UUID default/extension is used.

6. API Endpoints Delivered

Method	Path	Auth	Description
GET	/api/health	Public	Liveness check (Module 0)
POST	/api/auth/login	Public	Authenticate, set cookies
POST	/api/auth/refresh	Public (requires refresh cookie)	Rotate tokens
POST	/api/auth/logout	Authenticated	Revoke current session
GET	/api/auth/me	Authenticated	Current user profile
POST	/api/auth/change-password	Authenticated	Change password, revoke other sessions
POST	/api/auth/forgot-password	Public	Issue reset token (no email sent yet)
POST	/api/auth/reset-password	Public	Redeem reset token
POST	/api/users	ADMIN	Create user
GET	/api/users	ADMIN	Paginated user list
GET	/api/users/{id}	ADMIN	Get user by id
PUT	/api/users/{id}	ADMIN	Update user

Full request/response contracts are documented in API_DOCUMENTATION.md.

7. Testing Performed

Automated (backend, JUnit 5 + Mockito + Testcontainers)

29 tests, all passing, across six test classes:

Test class	Count	What it proves
JwtTokenProviderTest	4	Access tokens carry the expected claims; tampered, expired, and garbage tokens are all rejected
RefreshTokenServiceImplTest	6	Issue, rotation, reuse-detection (revokes the whole family), expiry rejection, unknown-token rejection, bulk revoke
AuthServiceImplTest	4	Login success issues tokens and audits; wrong password throws and audits a failure; change-password rejects a wrong current password; change-password success revokes all sessions and issues fresh ones
AuthControllerIntegrationTest	7	Full HTTP-level flow against a real Testcontainers PostgreSQL: login sets cookies, wrong password → 401, /me without a cookie → 401, /me with a valid cookie → 200, /me with an expired token → 401, refresh rotates and rejects reuse of the old token, logout revokes the refresh token
AuditLogServiceImplTest	2	Carried over from Module 0, still passing
InAppNotificationServiceImplTest	6	Carried over from Module 0, still passing

Run with `cd backend && mvn test` — **Tests run: 29, Failures: 0, Errors: 0.**

Manual / interactive (frontend)

The frontend project has no automated test runner configured (no Jest/React Testing Library/Playwright dependency committed to `package.json`). Verification was performed by:

1. `npm run lint` and `npm run build` — both clean.
2. A one-off Playwright/Chromium session (installed and run ad hoc, not part of the committed project) driving the live Docker stack through: login page load → login with the bootstrap admin → dashboard load → open the profile dropdown → change password with a wrong current password (expect rejection) → change password with the correct current password (expect success) → logout → re-login with the **new** password (proves the change persisted) → revert the password back. Every step passed; zero uncaught client-side exceptions.

This confirms the login → session → change-password → logout → re-login round trip works against the real backend and database, not just in isolation.

8. Bugs Found and Fixed

Profile dropdown crash (“This page couldn’t load”)

- **Symptom:** clicking the user icon in the topbar to open the account dropdown crashed the page with Next.js’s generic client-error screen.

- **Root cause:** the shadowed `DropdownMenuLabel` component (built on `@base-ui/react`'s `Menu.GroupLabel`) reads a React context that is only provided by `Menu.Group`. It was rendered directly inside `DropdownMenuContent` without a `DropdownMenuGroup` wrapper, so it threw `"Base UI: MenuGroupContext is missing..."` on render. With no `error.tsx` boundary in the app, Next.js's built-in fallback ("This page couldn't load") replaced the entire page.
- **Fix:** wrapped the label in `<DropdownMenuGroup>` in `frontend/src/components/layout/topbar.tsx`.
- **Verification:** confirmed via the `@base-ui/react` source (the exact invariant it throws), a codebase-wide search for any other unguarded use of the same pattern (none found), and the full Playwright session described in §7, which specifically re-opened the dropdown and asserted the crash screen was absent.

No other bugs were found during this module's implementation or verification.

9. Security Features Implemented

- BCrypt password hashing; passwords excluded from `toString()` and never returned by any endpoint.
- Signed JWT access tokens (HMAC-SHA256), 15-minute default TTL, re-validated against the live user record on every request (immediate effect for deactivation/suspension, not just token expiry).
- Opaque, database-backed refresh tokens (only the SHA-256 hash is stored) with rotation, reuse/theft detection (revokes the entire session family), and bulk invalidation on logout/password change.
- `HttpOnly` + configurable `Secure/SameSite` cookies; refresh cookie scoped to `/api/auth` only.
- Explicit-origin CORS with `allowCredentials(true)` (never a wildcard) as the CSRF mitigation, alongside `SameSite=Lax`.
- Role-based method security (`@PreAuthorize`) protecting `/api/users/**`.
- Consistent JSON 401/403 bodies at the security-filter level (`RestAuthenticationEntryPoint`, `RestAccessDeniedHandler`).
- Per-IP login rate limiting (`Bucket4j`, in-memory, 5/min).
- Bean Validation on every request DTO (email format, password strength, phone format, required fields).
- Full audit trail of login, failed login, logout, and password change events.
- No hardcoded credentials anywhere — the first admin account is created only from environment variables when the users table is empty.
- User enumeration resistance: `/api/auth/forgot-password` returns an identical response whether or not the email exists.

10. Verification Results

Check	Result
<code>mvn test</code> (backend)	✓ 29/29 passing
<code>npm run lint</code> (frontend)	✓ Clean
<code>npm run build</code> (frontend)	✓ Clean, all routes compiled
<code>docker compose up</code> (full stack)	✓ All 4 containers healthy/running
Flyway migrations V1 + V2	✓ Applied cleanly against a real Postgres container
<code>GET /api/health</code>	✓ Returns {"status": "UP"}
Swagger UI (/swagger-ui.html, dev profile)	✓ Loads
CORS credentialed request headers	✓ Access-Control-Allow-Origin + Access-Control-Allow-Credentials verified via curl
Cookie-based login/refresh/logout	✓ Verified via curl and a live browser session
Profile dropdown / change password / logout (browser)	✓ Verified via Playwright session after the bug fix

11. Current Project Status

- **Module 0:** complete, committed (dev branch, commit 6bbebc8).
- **Module 1:** complete and verified as described above; **not yet committed** to git.
- The full stack runs locally via `docker compose --env-file .env -f infra/docker-compose.yml up -d` on ports 3000 (frontend), 8080 (backend), 5433 (Postgres, host-mapped), 9000/9001 (MinIO).

12. Remaining Roadmap

Per the agreed build plan, not started yet:

2. Application Settings (full CRUD + admin UI over the AppSetting scaffold)
3. Media Upload (MinIO integration, upload endpoints, media library UI)
4. AI Content Generator + Prompt Templates
5. Content Draft Manager
6. Approval Workflow
7. Instagram Publisher
8. Website Publisher
9. Scheduler
10. Content History & Analytics
11. Activity Log Viewer (UI over the audit trail that has been recording since Module 0)
12. Notification Center (UI over the in-app notifications already being written)
13. Security Hardening & Finalization
14. DevOps & Docs Finalization

Also outstanding within the authentication surface itself: frontend forgot-password/reset-password pages (backend is ready; no UI yet), and real email delivery for password reset (currently the token is generated and persisted but never emailed — see the architecture doc's note on this).

System Architecture Diagrams — Arjun Sports AI Content Agent

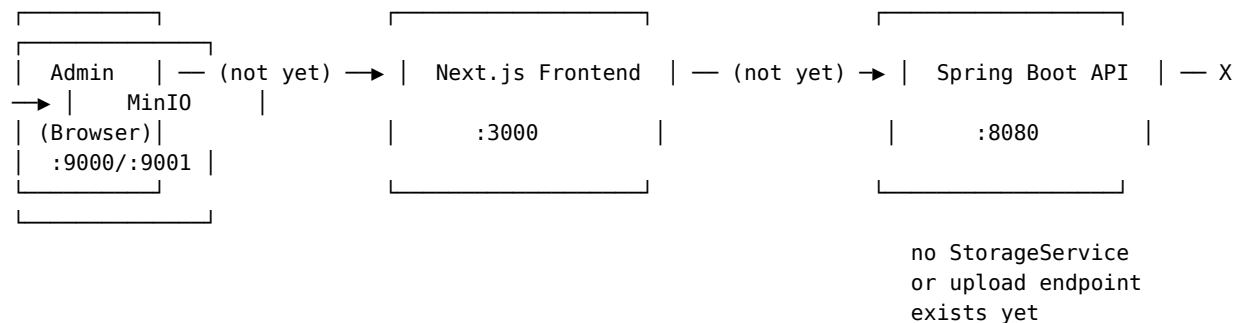
All diagrams below reflect Module 0 + Module 1 as implemented. Where a component is provisioned but not yet functionally wired up (MinIO), the diagram says so explicitly.

1. User → Frontend → Backend → PostgreSQL



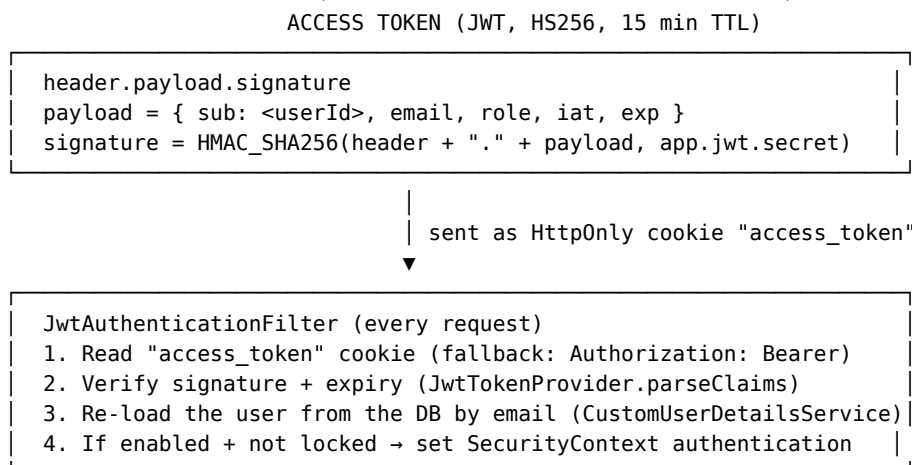
Explanation: the browser only ever talks to the Next.js server (port 3000). Next.js's server-side code (Server Components, proxy.ts) and the browser's own fetch calls both go on to the Spring Boot API (port 8080). Spring Data JPA/Hibernate is the only component that talks to PostgreSQL; the frontend has no database credentials or driver at all. Every row read from or written to PostgreSQL passes through a @Repository interface, then a @Service, then is shaped into a DTO before it ever becomes JSON.

2. User → Frontend → Backend → MinIO



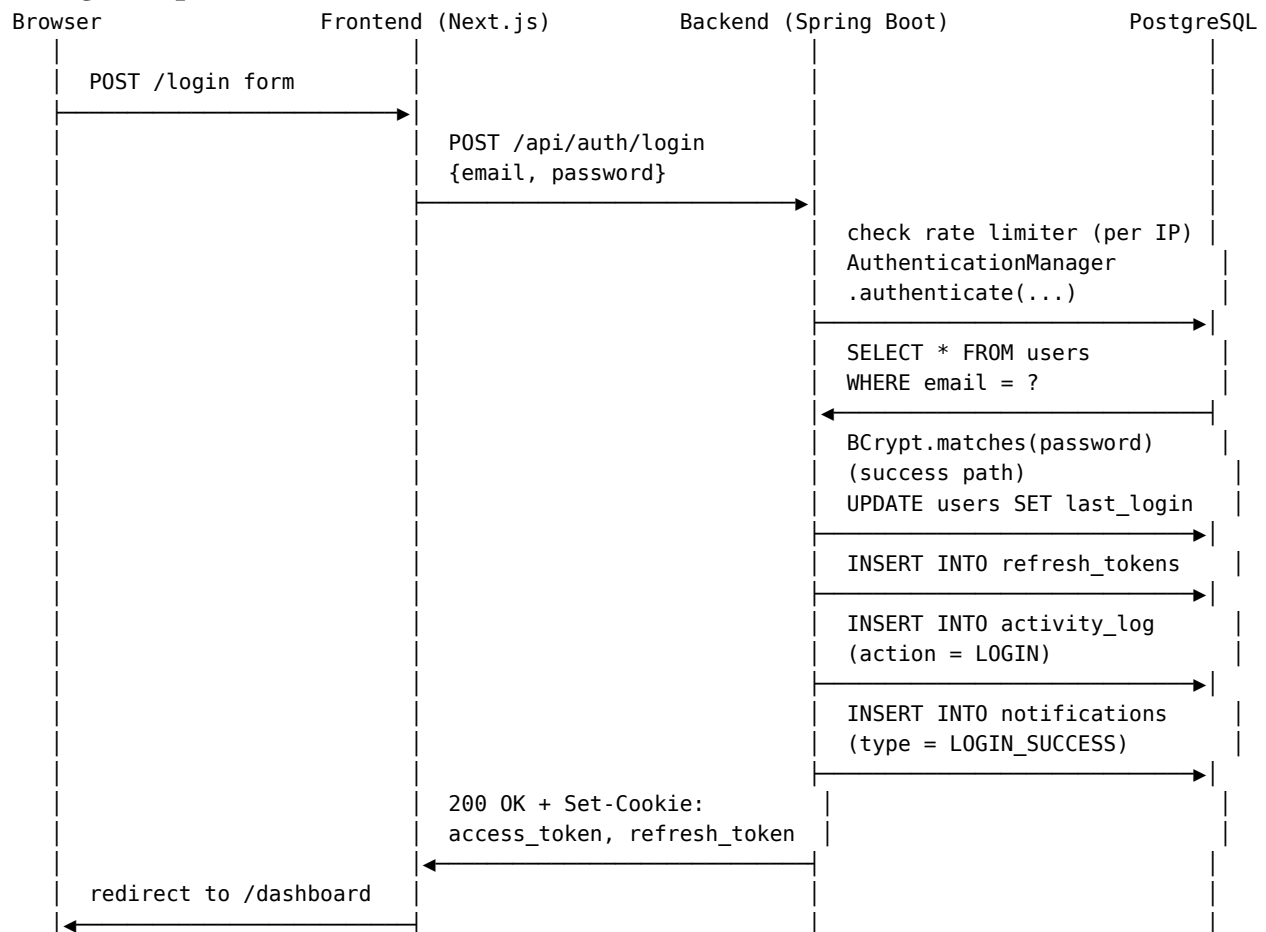
Explanation: MinIO runs as a container in docker-compose.yml and the io.minio:minio client library is a backend dependency, but **no code path currently connects to it** — there is no StorageService, no MinIO @Configuration class, and no upload controller. This diagram intentionally shows the gap: MinIO is infrastructure-ready for the future Media Upload module, not yet integrated.

3. JWT Authentication Flow (token structure and verification)



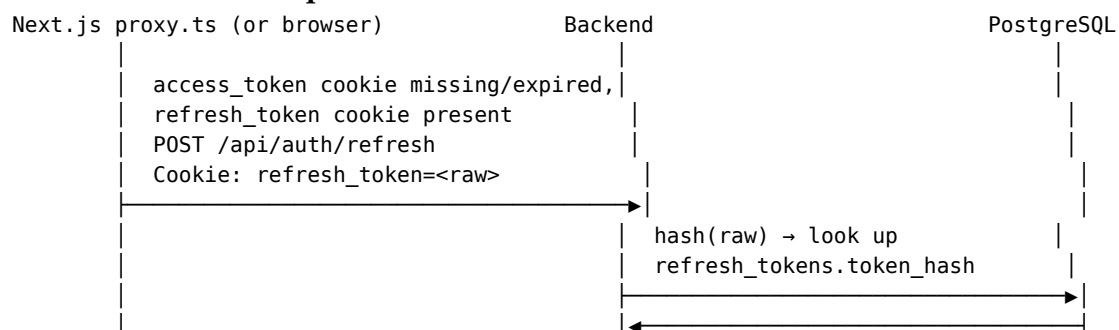
Explanation: the access token is stateless and self-contained (its claims are trusted once the signature verifies), but the filter deliberately re-fetches the user from PostgreSQL on every request instead of trusting the token's role/email claims for authorization decisions. This is a conscious trade-off: it costs one extra query per request but means suspending or deleting a user takes effect on their very next request, not up to 15 minutes later when their token would otherwise expire.

4. Login Sequence



Explanation: the login form posts directly to the backend (browser fetch with credentials: "include"), not through a Next.js API route. On failure, the same rate-limit check and audit step run, but instead of updating last_login/writing cookies, the backend writes a LOGIN_FAILED activity log entry and returns 401 with a generic message that does not reveal whether the email exists.

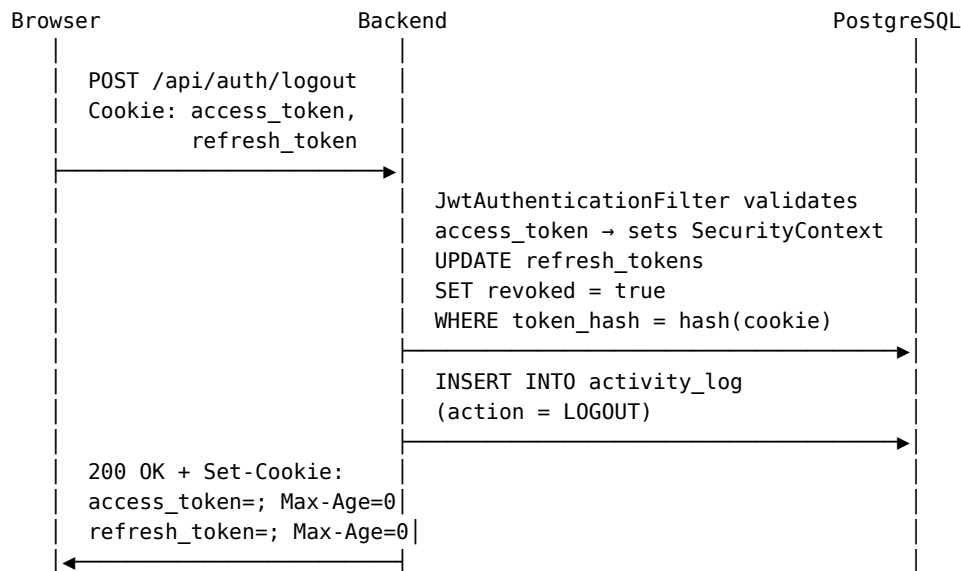
5. Refresh Token Sequence





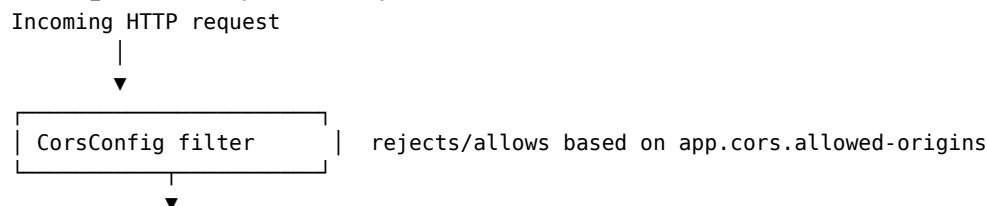
Explanation: this exact exchange happens two ways in this system — (a) explicitly, when the frontend calls `/api/auth/refresh` from `proxy.ts` on behalf of a visitor whose access token has expired but who still has a valid refresh cookie, and (b) it is the same code path a legitimate frontend “silent refresh” or a Postman/manual call would use. The reuse-detection branch (a revoked token being presented again) is a security control, not a normal-path outcome — it only fires if a refresh token is replayed after it has already been rotated away.

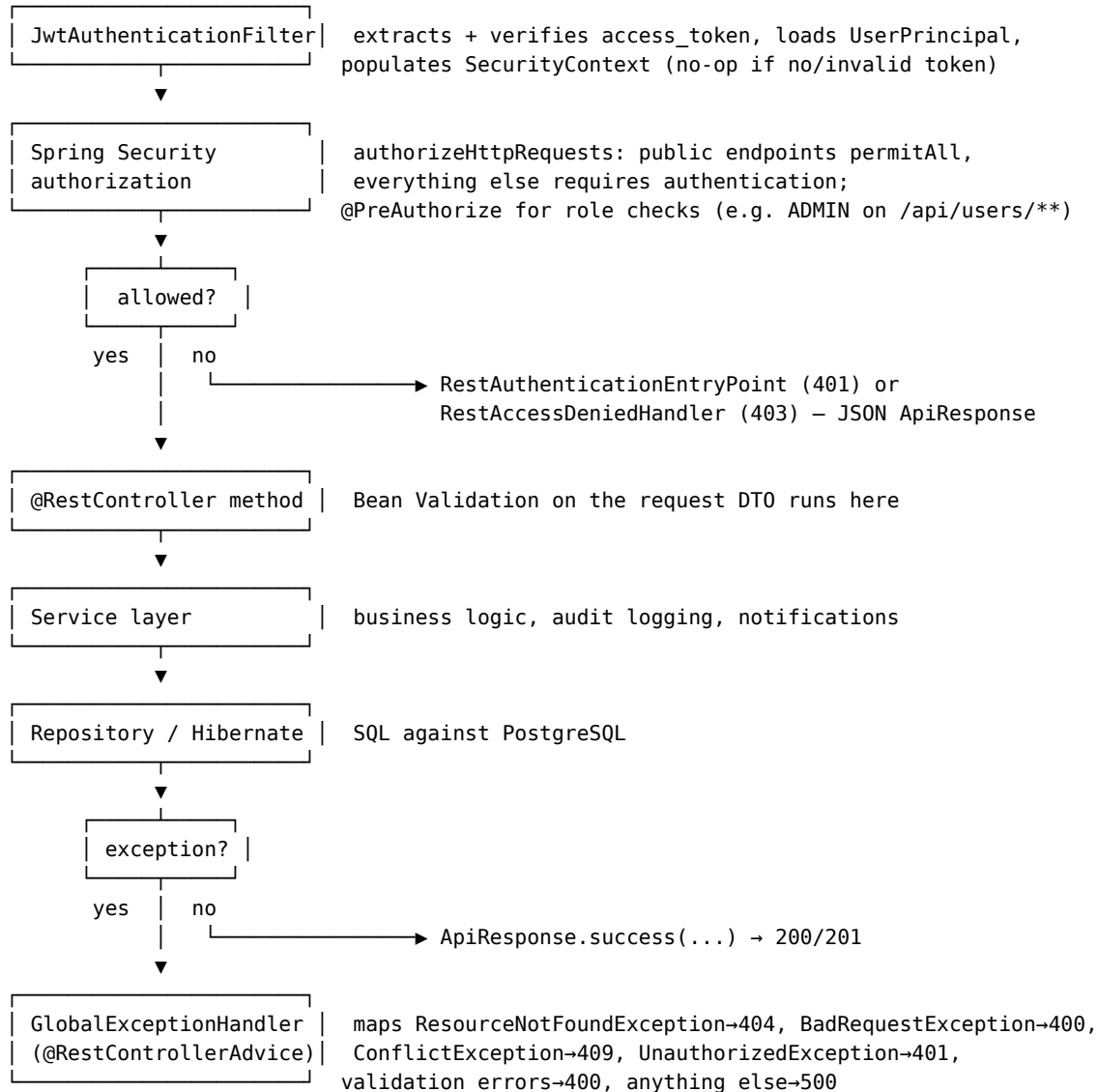
6. Logout Sequence



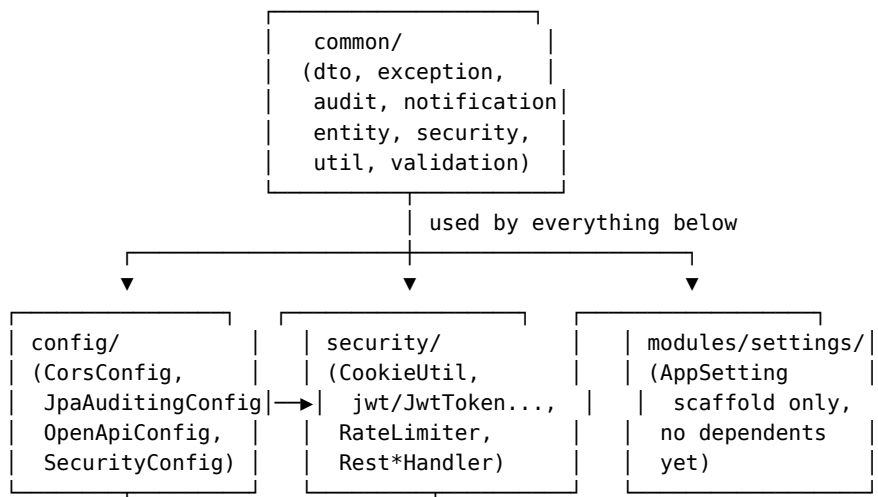
Explanation: logout only revokes the **single** refresh token presented in the request — other active sessions (e.g. a different browser) are left alone. Only a password change or reset revokes every session at once. The access token itself is never individually invalidated server-side (it is stateless); logout relies on the cookie being cleared client-side and the token’s own short (15-minute) expiry to close the window.

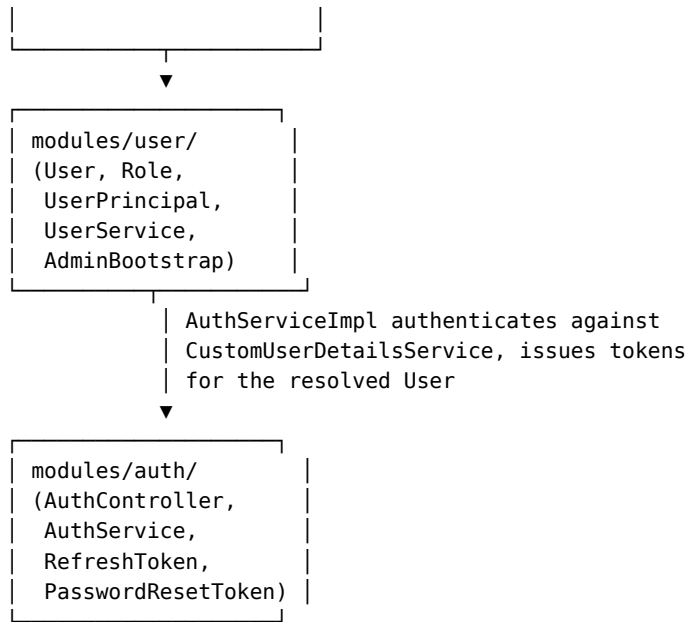
7. Request Lifecycle (every API call)





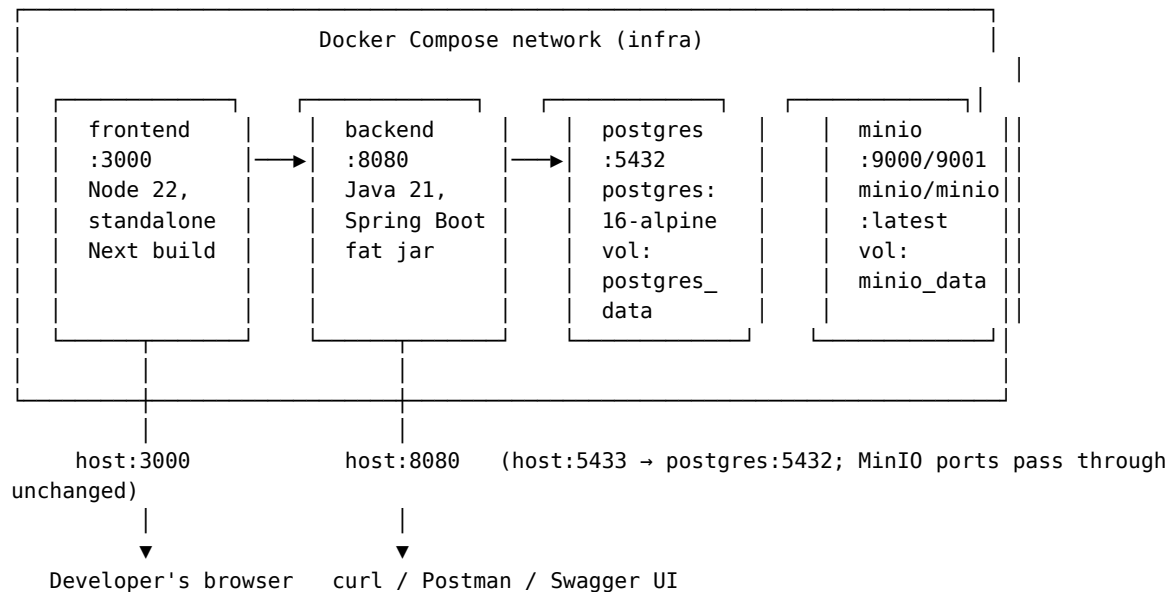
8. Module Dependency Diagram





Explanation: common has no dependency on any feature module — it is the shared foundation. modules/auth depends on modules/user (it authenticates and issues tokens *for* a User), never the other way around. modules/settings is a standalone scaffold with no other module depending on it yet. This ordering is why Module 1 could safely reuse AuditLogService/NotificationService without editing them — a new module can depend “downward” on common and, if needed, on modules/user, without the reverse ever being true.

9. Deployment Architecture



Explanation: depends_on in docker-compose.yml ensures Postgres reports healthy (via pg_isready) before the backend container starts, and MinIO has at least started (no health check defined for it yet, since nothing consumes it). The frontend depends on the backend starting. Postgres’s host-exposed port is deliberately remapped to 5433 by default to avoid colliding with any other local Postgres instance; internally, on the Compose network, it is still reached at postgres:5432. The frontend container reaches the backend at http://backend:8080/api for its own

server-side calls, while the developer's actual browser reaches it at `http://localhost:8080/api` — these are two different URLs for two different callers, both pointing at the same backend container.

API Documentation — Arjun Sports AI Content Agent

Base URL (local): `http://localhost:8080/api` **Scope:** only endpoints that exist in the codebase today (Module 0 health check + Module 1 auth/user endpoints). Interactive documentation is also available at `/swagger-ui.html` and `/api-docs` when the dev Spring profile is active (both are disabled in the prod profile).

Response Envelope

Every endpoint — success or failure — returns the same JSON shape (`ApiResponse<T>`). Fields with a null value are omitted from the JSON (`@JsonInclude(NON_NULL)`), so a success response typically has no errors key and an error response typically has no data key.

```
{
  "success": true,
  "message": "Optional human-readable message",
  "data": { "...": "endpoint-specific payload" },
  "errors": ["Optional list of field-level validation errors"],
  "timestamp": "2026-07-07T06:29:25.682Z"
}
```

List endpoints wrap a `PageResponse<T>` inside data:

```
{
  "content": [ { "...": "item" } ],
  "page": 0,
  "size": 20,
  "totalElements": 1,
  "totalPages": 1,
  "last": true
}
```

Authentication Requirements

- **Public** endpoints require no credentials.
- **Authenticated** endpoints require a valid `access_token` cookie (set automatically by `/api/auth/login` or `/api/auth/refresh`), or an `Authorization: Bearer <token>` header as a fallback for non-browser clients.
- **ADMIN** endpoints additionally require the authenticated user's role to be `ADMIN` (the only role that exists today).
- All cookies are `HttpOnly`; the refresh cookie is further scoped to the `/api/auth` path only, so it is never sent to non-auth endpoints.

Status Codes Used

Code	Meaning	Where it comes from
200	Success	Normal responses
201	Created	POST /api/users
400	Bad request / validation failure	GlobalExceptionHandler (Bean Validation, BadRequestException)
401	Unauthorized	Missing/invalid/expired token, wrong credentials, UnauthorizedException, Spring Security's RestAuthenticationEntryPoint
403	Forbidden	Authenticated but wrong role — RestAccessDeniedHandler
404	Not found	ResourceNotFoundException
409	Conflict	ConflictException (e.g. duplicate email)
429	Too many requests	Login rate limiter exceeded
500	Internal server error	Any uncaught exception (GlobalExceptionHandler fallback)

Error Response Shape

```
{
  "success": false,
  "message": "Validation failed",
  "errors": [
    "email: Email must be a valid address",
    "password: Password is required"
  ],
  "timestamp": "2026-07-07T06:29:25.682Z"
}
```

A non-validation error omits errors entirely:

```
{
  "success": false,
  "message": "Invalid email or password",
  "timestamp": "2026-07-07T06:29:25.682Z"
}
```

Health API

GET /api/health

Public. Liveness check.

Response 200:

```
{
  "success": true,
```

```

    "data": {
      "status": "UP",
      "service": "content-agent-backend",
      "timestamp": "2026-07-07T06:29:25.682Z"
    },
    "timestamp": "2026-07-07T06:29:25.682Z"
  }
}

```

Authentication APIs

POST /api/auth/login

Public. Authenticates a user and sets the access_token and refresh_token HttpOnly cookies on success.

Request:

```

{
  "email": "admin@arjunsports.example",
  "password": "DevAdmin123"
}

```

Validation: email required + valid email format; password required.

Response 200 (also sets Set-Cookie: access_token=... and Set-Cookie: refresh_token=...):

```

{
  "success": true,
  "message": "Login successful",
  "data": {
    "id": "3f7ble2a-...",
    "firstName": "Academy",
    "lastName": "Admin",
    "email": "admin@arjunsports.example",
    "phone": null,
    "role": "ADMIN",
    "status": "ACTIVE",
    "active": true,
    "lastLogin": "2026-07-07T06:29:25.682Z",
    "createdAt": "2026-07-01T10:00:00Z",
    "updatedAt": "2026-07-07T06:29:25.682Z"
  },
  "timestamp": "2026-07-07T06:29:25.682Z"
}

```

Errors: - 401 — wrong email or password (identical message for both, to avoid revealing which one was wrong):
 json { "success": false, "message": "Invalid email or password", "timestamp": "..." } - 429 — more than 5 attempts from the same client IP within a minute: json { "success": false, "message": "Too many login attempts. Please try again in a minute.", "timestamp": "..." }

POST /api/auth/refresh

Public (but requires a valid refresh_token cookie — there is no request body). Rotates the refresh token and issues a new access token.

Response 200: same shape as login, with fresh Set-Cookie headers for both tokens.

Errors: - 401 — no refresh token cookie present, token unknown/expired, or the token had already been rotated out (reuse detection — in this case every session for the user is also revoked server-side).

POST /api/auth/logout

Authenticated. Revokes the refresh token presented in the request and clears both cookies.

Response 200:

```
{ "success": true, "message": "Logged out", "timestamp": "..." }
```

GET /api/auth/me

Authenticated. Returns the profile of the currently authenticated user.

Response 200: same user object shape as the login response's data.

Errors: 401 if the access token is missing, invalid, or expired.

POST /api/auth/change-password

Authenticated. Changes the current user's password and revokes every other active session (a fresh token pair is issued for the current session, so the caller stays logged in).

Request:

```
{
  "currentPassword": "DevAdmin123",
  "newPassword": "NewPassw0rd123"
}
```

Validation: currentPassword required; newPassword required, must match the password-strength pattern (minimum 8 characters, at least one letter and one digit).

Response 200: same user object shape, plus fresh Set-Cookie headers.

Errors: - 400 — currentPassword does not match the stored password: json { "success": false, "message": "Current password is incorrect", "timestamp": "..."} - 400 — newPassword fails the strength pattern (validation error list as shown above).

POST /api/auth/forgot-password

Public. Always returns the same response whether or not the email exists, to prevent account enumeration. If the email does exist, a password reset token is generated and persisted (hashed) with a 1-hour expiry.

Request:

```
{ "email": "admin@arjunsports.example" }
```

Response 200 (always, regardless of whether the account exists):

```
{
  "success": true,
  "message": "If an account exists for this email, password reset instructions will be sent",
  "timestamp": "...
}
```

Note: email delivery is not implemented yet. The token is generated and stored but never emailed to the user in the current implementation — see IMPLEMENTATION_REPORT.md §12.

POST /api/auth/reset-password

Public. Redeems a password reset token issued by forgot-password.

Request:

```
{
  "token": "<raw token value>",
}
```

```
    "newPassword": "NewPassw0rd123"
}
```

Response 200:

```
{ "success": true, "message": "Password reset successfully", "timestamp": "..." }
```

Errors: 400 if the token is unknown, already used, or expired:

```
{ "success": false, "message": "Invalid or expired reset token", "timestamp": "..." }
```

User Management APIs (ADMIN only)

All endpoints under `/api/users/**` require the authenticated user's role to be ADMIN. A non-admin authenticated user gets 403; an unauthenticated caller gets 401.

POST /api/users

Creates a new user.

Request:

```
{
  "firstName": "Jane",
  "lastName": "Coach",
  "email": "jane.coach@arjunsports.example",
  "phone": "+919876543210",
  "password": "CoachPass123",
  "role": "ADMIN"
}
```

Validation: firstName/lastName required (max 100 chars); email required, valid format; phone optional but must match `^\+?[1-9]\d{7,14}$` if present; password required, must match the password-strength pattern; role required (ADMIN is the only value that exists today).

Response 201:

```
{
  "success": true,
  "message": "User created",
  "data": {
    "id": "9c2e...",
    "firstName": "Jane",
    "lastName": "Coach",
    "email": "jane.coach@arjunsports.example",
    "phone": "+919876543210",
    "role": "ADMIN",
    "status": "ACTIVE",
    "active": true,
    "lastLogin": null,
    "createdAt": "...",
    "updatedAt": "..."
  },
  "timestamp": "..."
}
```

Errors: 409 if the email is already registered.

GET /api/users

Paginated user list. Accepts standard Spring Data Pageable query parameters: page, size, sort (e.g. `GET /api/users?page=0&size=20&sort=lastName,asc`).

Response 200: data is a PageResponse<UserResponse> (see the envelope section above).

GET /api/users/{id}

Response 200: a single UserResponse. **Errors:** 404 if the id does not exist.

PUT /api/users/{id}

Updates a user's profile, role, status, and active flag (email and password are not editable through this endpoint).

Request:

```
{
  "firstName": "Jane",
  "lastName": "Coach",
  "phone": "+919876543210",
  "role": "ADMIN",
  "status": "ACTIVE",
  "active": true
}
```

Response 200: the updated UserResponse. **Errors:** 404 if the id does not exist; 400 on validation failure.

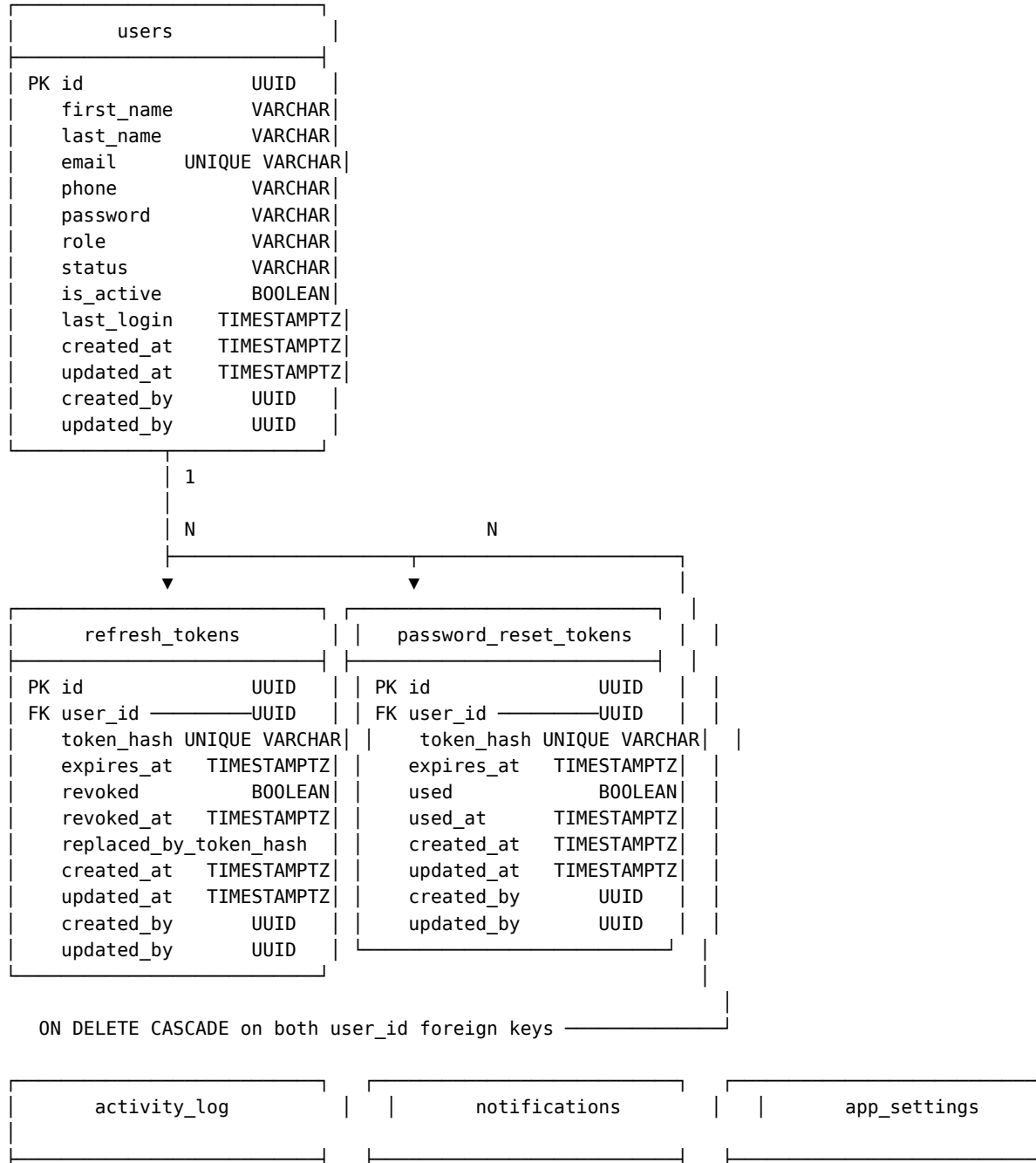
Database Design — Arjun Sports AI Content Agent

Engine: PostgreSQL 16. **Schema ownership:** Flyway migrations only (spring.jpa.hibernate.ddl-auto: validate — Hibernate never generates or alters DDL, it only verifies entity mappings match what Flyway created).

Primary keys: every table uses a UUID generated application-side by Hibernate (@UuidGenerator), so no table relies on a database-side default or the pgcrypto/uuid-oss extensions.

This document covers exactly the six tables that exist after migrations V1 and V2. No table described here is speculative.

1. Entity-Relationship Diagram (ASCII)



PK id	UUID	PK id	UUID	PK id	UUID
actor_id	UUID	recipient_id	UUID	setting_key	UNIQUE
VARCHAR					
actor_email	VARCHAR	type	VARCHAR	setting_value	TEXT
action	VARCHAR	title	VARCHAR	category	VARCHAR
entity_type	VARCHAR	message	TEXT	description	VARCHAR
entity_id	UUID	link	VARCHAR	is_secret	BOOLEAN
metadata	TEXT	is_read	BOOLEAN	created_at	TIMESTAMPTZ
created_at	TIMESTAMPTZ	read_at	TIMESTAMPTZ	updated_at	TIMESTAMPTZ
(no FK - actor_id/entity_id		created_at	TIMESTAMPTZ	created_by	UUID
are denormalized, not		updated_at	TIMESTAMPTZ	updated_by	UUID
relational references)		created_by	UUID		
		updated_by	UUID		
(Module 0 - written to by				(Module 0 scaffold - no	
Module 1's auth events)				service/controller yet)	
		(Module 0 - written to by			
		Module 1's auth events)			

Why `activity_log` and `notifications` have no foreign keys to users: both are intentionally denormalized. `activity_log.actor_id/actor_email` and `notifications.recipient_id` store the user identifier and (for the log) email *as they were at the time of the event*, so the audit trail and notification history remain meaningful even if a user is later deleted — a hard FK with `ON DELETE CASCADE` would silently erase history, and `ON DELETE SET NULL` would lose the actor's identity. This is a deliberate design choice, not an oversight.

2. Tables

users (Module 1)

Column	Type	Constraints
id	UUID	PRIMARY KEY (app-generated)
first_name	VARCHAR(100)	NOT NULL
last_name	VARCHAR(100)	NOT NULL
email	VARCHAR(255)	NOT NULL, UNIQUE
phone	VARCHAR(20)	nullable
password	VARCHAR(255)	NOT NULL (BCrypt hash, never plaintext)
role	VARCHAR(30)	NOT NULL (ADMIN today; enum-backed, extensible)
status	VARCHAR(30)	NOT NULL, DEFAULT 'ACTIVE' (ACTIVE, INACTIVE, SUSPENDED)
is_active	BOOLEAN	NOT NULL, DEFAULT TRUE
last_login	TIMESTAMPTZ	nullable
created_at, updated_at	TIMESTAMPTZ	NOT NULL (JPA auditing)
created_by, updated_by	UUID	nullable (JPA auditing — null until a second user exists to act)

refresh_tokens (Module 1)

Column	Type	Constraints
id	UUID	PRIMARY KEY
user_id	UUID	NOT NULL, FK → users(id) ON DELETE CASCADE
token_hash	VARCHAR(128)	NOT NULL, UNIQUE (SHA-256 hex digest; raw token is never stored)
expires_at	TIMESTAMPTZ	NOT NULL
revoked	BOOLEAN	NOT NULL, DEFAULT FALSE
revoked_at	TIMESTAMPTZ	nullable
replaced_by_token_hash	VARCHAR(128)	nullable — links a rotated-out row to the row that replaced it
created_at, updated_at, created_by, updated_by	—	JPA auditing columns

password_reset_tokens (Module 1)

Column	Type	Constraints
id	UUID	PRIMARY KEY
user_id	UUID	NOT NULL, FK → users(id) ON DELETE CASCADE
token_hash	VARCHAR(128)	NOT NULL, UNIQUE (SHA-256 hex digest)
expires_at	TIMESTAMPTZ	NOT NULL (1 hour from issuance)
used	BOOLEAN	NOT NULL, DEFAULT FALSE
used_at	TIMESTAMPTZ	nullable
created_at, updated_at, created_by, updated_by	—	JPA auditing columns

activity_log (Module 0, populated by Module 1)

Column	Type	Constraints
id	UUID	PRIMARY KEY
actor_id	UUID	nullable (null for unauthenticated/ system events, e.g. a failed login for an unknown email)
actor_email	VARCHAR(255)	nullable
action	VARCHAR(40)	NOT NULL — enum ActivityAction: CREATE, UPDATE, DELETE, UPLOAD, GENERATE, APPROVE, REJECT, PUBLISH, SCHEDULE, LOGIN, LOGIN_FAILED, LOGOUT, PASSWORD_CHANGE, SETTINGS_CHANGE, OTHER
entity_type	VARCHAR(100)	NOT NULL (e.g. "User")
entity_id	UUID	nullable
metadata	TEXT	nullable — JSON-serialized context (e.g. {"clientId": "..."})
created_at	TIMESTAMPTZ	NOT NULL

No updated_at/updated_by: audit entries are immutable once written, so update-tracking columns do not apply.

notifications (Module 0, populated by Module 1)

Column	Type	Constraints
id	UUID	PRIMARY KEY
recipient_id	UUID	nullable (null = broadcast to all admins)
type	VARCHAR(40)	NOT NULL — enum NotificationType: LOGIN_SUCCESS, PASSWORD_CHANGED, GENERATION_COMPLETED, APPROVAL_REQUIRED, APPROVED, REJECTED, PUBLISH_SUCCESS, PUBLISH_FAILURE, SCHEDULE_COMPLETED, SCHEDULE_FAILED, SYSTEM
title	VARCHAR(255)	NOT NULL
message	TEXT	NOT NULL
link	VARCHAR(500)	nullable (deep link for a future notification-center UI)
is_read	BOOLEAN	NOT NULL, DEFAULT FALSE
read_at	TIMESTAMPTZ	nullable
created_at, updated_at, created_by, updated_by	—	JPA auditing columns

Only LOGIN_SUCCESS and PASSWORD_CHANGED are actually written today (by Module 1). The other enum values are reserved for later modules.

app_settings (Module 0 scaffold — no service/controller yet)

Column	Type	Constraints
id	UUID	PRIMARY KEY
setting_key	VARCHAR(150)	NOT NULL, UNIQUE
setting_value	TEXT	nullable
category	VARCHAR(40)	NOT NULL — enum SettingCategory: AI_PROVIDER, GENERATION, SCHEDULING, STORAGE, INTEGRATION
description	VARCHAR(500)	nullable
is_secret	BOOLEAN	NOT NULL, DEFAULT FALSE
created_at, updated_at, created_by, updated_by	—	JPA auditing columns

This table has no rows written by any current code path — it exists so a future Application Settings module can build its CRUD layer directly on top of an already-migrated schema.

3. Relationships

Relationship	Cardinality	Enforcement
users → refresh_tokens	1 → N	Database FK, ON DELETE CASCADE
users → password_reset_tokens	1 → N	Database FK, ON DELETE CASCADE
users → activity_log	logical 1 → N (by actor_id)	Not a database FK — denormalized by design (see §1)
users → notifications	logical 1 → N (by recipient_id)	Not a database FK — denormalized by design (see §1)

app_settings has no relationships to any other table.

4. Constraints

- users.email, refresh_tokens.token_hash, password_reset_tokens.token_hash, and app_settings.setting_key all carry a UNIQUE constraint.
- refresh_tokens.user_id and password_reset_tokens.user_id are NOT NULL foreign keys with ON DELETE CASCADE — deleting a user immediately invalidates all of their tokens.
- All boolean flags (is_active, revoked, used, is_read, is_secret) are NOT NULL with an explicit DEFAULT.
- All timestamp columns use TIMESTAMPTZ (timezone-aware), and Hibernate is configured with hibernate.jdbc.time_zone: UTC so every stored instant is unambiguous.

5. Indexes

Index	Table	Columns	Purpose
idx_users_email	users	email	Login lookup (findByEmailIgnoreCase)
idx_refresh_tokens_user	refresh_tokens	user_id, revoked	“all active tokens for user” queries (logout-all, password change)
idx_refresh_tokens_hash	refresh_tokens	token_hash	Refresh/rotate lookup by presented token
idx_password_reset_tokens_user	password_reset_tokens	user_id	Future lookups by user
idx_activity_log_entity	activity_log	entity_type, entity_id	“history for this record” queries
idx_activity_log_actor	activity_log	actor_id	“history by this user” queries
idx_activity_log_action	activity_log	action	Filtering by action type
idx_activity_log_created_at	activity_log	created_at DESC	Recent-first listing
idx_notifications_recipient_unread	notifications	recipient_id, is_read	Unread-count / inbox queries
idx_notifications_created_at	notifications	created_at DESC	Recent-first listing
idx_app_settings_category	app_settings	category	Grouped settings retrieval

Every index above backs an actual repository query method that exists in the codebase today (e.g. `UserRepository.findByEmailIgnoreCase`, `RefreshTokenRepository.findByIdAndRevokedFalse`) — none are speculative.

6. Flyway Migrations

Version	File	Module	Contents
V1	<code>V1__init_schema.sql</code>	Module 0	<code>activity_log</code> , <code>notifications</code> , <code>app_settings</code> + their indexes
V2	<code>V2__auth_and_users.sql</code>	Module 1	<code>users</code> , <code>refresh_tokens</code> , <code>password_reset_tokens</code> + their indexes and foreign keys

Both have been applied successfully against a real PostgreSQL 16 instance (verified via `docker compose up` and via the Testcontainers-backed `AuthControllerIntegrationTest`, which boots a disposable Postgres container and runs both migrations before any test executes).

7. Future Database Roadmap

Not yet designed or migrated — listed here only to show what the current schema deliberately leaves room for, not as a commitment to a specific column layout:

- **Media** table(s) for the Media Upload module (file metadata, MinIO object keys, upload type).
- **Content draft** table(s) for AI-generated content (versioned, linked to media).
- **Approval** state/audit table(s) for the approval workflow (may reuse `activity_log` rather than a new table).
- **Publish record** table(s) for Instagram/website publishing history.
- **Schedule** table(s) for the scheduler module.
- Full CRUD usage of the existing `app_settings` table by the Application Settings module.
- Possible introduction of additional `Role` enum values (e.g. `EDITOR`) — the `users.role` column already supports this without a migration, since it is a plain `VARCHAR` validated at the application layer, not a database `CHECK/ENUM` type.